

Yes — I can absolutely help you build this evaluator tool, and your chosen approach is **exactly the right one**.

What you're proposing is essentially a **mini automated grading platform**, and with a few design decisions, we can make it:

- Reliable
- Fast for evaluation
- Easy to maintain across all 4 sets

Final Evaluation Architecture (What we will build)

◆ Input

- Student submits:
 - Vercel URL

◆ Evaluator Tool

- Loads the app
- Navigates routes
- Reads:
 - DOM (data-testid)
 - window.appState

◆ Output

```
{  
  "Q1": "Pass",  
  "Q2": "Fail",  
  "Q3": "Pass",  
  "Q4": "Pass",  
  "Q5": "Fail",  
  "Score": 68  
}
```

Tech Stack (Recommended)

Use:

- **Playwright (Node.js)**

Why Playwright:

- Works with deployed URLs
- Fast and reliable
- Supports navigation, clicks, DOM extraction
- Easy scripting

What We Need to Standardize (VERY IMPORTANT)

Before coding evaluator, your question paper must enforce:

1. Fixed Routes (you already decided)

```
/orders  
/orders/:id  
/filter  
/stats
```

2. Required Test IDs

We will define this in question paper:

```
<div data-testid="total-orders"></div>  
<div data-testid="delivered-orders"></div>  
<div data-testid="cancelled-orders"></div>
```

3. Required Global State Exposure

```
window.appState = {  
  totalOrders,  
  deliveredOrders,  
  cancelledOrders  
};
```

 This is your **evaluation backbone**

Evaluator Tool – Phase Plan

Phase 1: Basic Evaluator (I'll help you build this first)

Features:

- Accept URL
- Open site
- Navigate pages
- Read DOM values
- Print results

◆ Phase 2: Smart Evaluation

- Compare expected vs actual
- Score calculation
- Error reporting

◆ Phase 3: UI Tool (Optional)

- Input box (paste URL)
- Button: Evaluate
- Show result table

Sample Playwright Script (Starter)

Here's a working base:

```
import { chromium } from "playwright";
```

```
async function evaluateApp(url) {  
  const browser = await chromium.launch();  
  const page = await browser.newPage();
```

```
  const result = {  
    Q1: "Fail",  
    Q2: "Fail",  
    Q3: "Fail",  
    Q4: "Fail",  
    Q5: "Fail",  
  };
```

```
  try {  
    // Open app  
    await page.goto(url);  
  
    // Go to stats page  
    await page.goto(`${url}/stats`);
```

```
    // Read values  
    const totalOrders = await page
```

```
.locator('[data-testid="total-orders"]')
.textContent();

const deliveredOrders = await page
  .locator('[data-testid="delivered-orders"]')
  .textContent();

if (totalOrders && deliveredOrders) {
  result.Q5 = "Pass";
}

// Access global state
const appState = await page.evaluate(() => window.appState);

if (appState && appState.totalOrders) {
  result.Q1 = "Pass";
}

} catch (err) {
  console.log("Error:", err.message);
}

await browser.close();
return result;
}

// Example usage
evaluateApp("https://student-app.vercel.app").then(console.log);
```

How We Will Design Questions (Important)

Now your question paper must:

- Force use of:
 - Context
 - Reducer
 - Derived state
- AND expose:
 - window.appState

- data-testid

⚠ Reality Check

What automation can check:

- ✓ Counts
- ✓ Filtering
- ✓ Navigation
- ✓ Presence of UI

What it cannot fully check:

- ✗ Code quality
- ✗ Reducer purity
- ✗ Clean architecture

👉 So:

You still keep:

- 80% automated
- 20% manual review

🧠 Final Recommendation

You are building something very strong:

“Semi-automated corporate-style evaluation system”

🚀 What I'll Do Next (If You Want)

I can help you step-by-step:

1. Final SET A Question Paper (with test IDs + rules)
2. Full Playwright Evaluator (production ready)
3. Scoring logic
4. Reusable template for SET B, C, D

✅ Next Step

Say:

👉 “build evaluator step by step”

or



👉 “create SET A question paper (testable)”

We’ll build this system properly 🔥